Substituting the Last-to-First mapping with count arrays

If you implemented **BWMATCHING** in the previous lesson, you probably found the algorithm to be slow. The reason for its sluggishness is that updating the pointers *top* and *bottom* is time-intensive, since it requires examining every symbol in *LastColumn* between *top* and *bottom*. To improve **BWMATCHING**, we introduce a function $Count_{symbol}(i, LastColumn)$, which returns the number of occurrences of *symbol* in the first *i* positions of *LastColumn*. For example, $Count_{\text{"n"}}(10, \texttt{"smnpbnnaaaaa\$a"}) = 3$, and $Count_{\text{"a"}}(4, \texttt{"smnpbnnaaaaa\$a"}) = 0$. Below, we show arrays holding $Count_{symbol}(i, \texttt{"smnpbnnaaaaa\$a"})$ for every *symbol* occurring in `"panamabananas$"`.

i	*FirstColumn*	*LastColumn*	LASTTOFIRST(i)	COUNT						
				$	a	b	m	n	p	s
0	$\$_1$	s_1	13	0	0	0	0	0	0	0
1	a_1	m_1	8	0	0	0	0	0	0	1
2	a_2	n_1	9	0	0	0	1	0	0	1
3	a_3	p_1	12	0	0	0	1	1	0	1
4	a_4	b_1	7	0	0	0	1	1	1	1
5	a_5	n_2	10	0	0	1	1	1	1	1
6	a_6	n_3	11	0	0	1	1	2	1	1
7	b_1	a_1	1	0	0	1	1	3	1	1
8	m_1	a_2	2	0	1	1	1	3	1	1
9	n_1	a_3	3	0	2	1	1	3	1	1
10	n_2	a_4	4	0	3	1	1	3	1	1
11	n_3	a_5	5	0	4	1	1	3	1	1
12	p_1	$\$_1$	0	0	5	1	1	3	1	1
13	s_1	a_6	6	1	5	1	1	3	1	1
				1	6	1	1	3	1	1

STOP and Think: Compute the *Count* arrays for *BWT*(`"abracadabra$"`).

We will say that the *k*-th occurrence of *symbol* in a column of a matrix has **rank** *k* in this column. For *Text* = `"panamabananas$"`, looking again at the table from the previous step (reproduced below), note that the first and last occurrences of *symbol* in the range of positions from *top* to *bottom* in *LastColumn* have respective ranks

$$Count_{symbol}(top, LastColumn) + 1$$

and

$$Count_{symbol}(bottom + 1, LastColumn).$$

For example, when *top* = 1, *bottom* = 6, and *symbol* = `"n"`, we have $Count_{\texttt{"n"}}(top1, LastColumn) + 1 = 1$ and $Count_{\texttt{"n"}}(bottom + 1, LastColumn) = 3$. The occurrences of `"n"` having ranks 1 and 3 are located at positions 2 and 6 of *LastColumn*, implying that we should update *top* to *LastToFirst*(2) = 9 and *bottom* to *LastToFirst*(6) = 11.

i	*FirstColumn*	*LastColumn*	LastToFirst(i)	Count						
				$	a	b	m	n	p	s
0	$$_1$	s$_1$	13	0	0	0	0	0	0	0
1	a$_1$	m$_1$	8	0	0	0	0	0	0	1
2	a$_2$	n$_1$	9	0	0	0	1	0	0	1
3	a$_3$	p$_1$	12	0	0	0	1	1	0	1
4	a$_4$	b$_1$	7	0	0	0	1	1	1	1
5	a$_5$	n$_2$	10	0	0	1	1	1	1	1
6	a$_6$	n$_3$	11	0	0	1	1	2	1	1
7	b$_1$	a$_1$	1	0	0	1	1	3	1	1
8	m$_1$	a$_2$	2	0	1	1	1	3	1	1
9	n$_1$	a$_3$	3	0	2	1	1	3	1	1
10	n$_2$	a$_4$	4	0	3	1	1	3	1	1
11	n$_3$	a$_5$	5	0	4	1	1	3	1	1
12	p$_1$	$$_1$	0	1	4	1	1	3	1	1
13	s$_1$	a$_6$	6	1	5	1	1	3	1	1
				1	6	1	1	3	1	1

As a result of the discussion on the previous step, let's revisit the pseudocode for **BWMATCHING**.

```
BWMATCHING(FirstColumn, LastColumn, Pattern, LastToFirst)
  top ← 0
  bottom ← |LastColumn| − 1
  while top ≤ bottom
    if Pattern is nonempty
      symbol ← last letter in Pattern
      remove last letter from Pattern
      if positions from top to bottom in LastColumn contain an occurrence of symbol
        topIndex ← first position where symbol occurs among positions where symbol
                   occurs from top to bottom in LastColumn
        bottomIndex ← last position of symbol among positions from top to bottom in
                      LastColumn
        top ← LastToFirst(topIndex)
        bottom ← LastToFirst(bottomIndex)
      else
        return 0
    else
      return bottom − top + 1
```

The four green lines can be rewritten as follows:

```
topIndex ← position of symbol with rank Count_symbol(top, LastColumn) + 1 in LastColumn
bottomIndex ← position of symbol with rank Count_symbol(bottom + 1, LastColumn) in
              LastColumn
top ← LastToFirst(topIndex)
bottom ← LastToFirst(bottomIndex)
```

Let's examine these four lines of pseudocode once again.

topIndex ← position of *symbol* with rank $Count_{symbol}(top, LastColumn) + 1$ in *LastColumn*
bottomIndex ← position of *symbol* with rank $Count_{symbol}(bottom + 1, LastColumn)$ in *LastColumn*
top ← *LastToFirst*(*topIndex*)
bottom ← *LastToFirst*(*bottomIndex*)

By eliminating the variables *topIndex* and *bottomIndex*, we can reduce these four lines to only two lines:

top ← *LastToFirst*(position of *symbol* with rank $Count_{symbol}(top, LastColumn) + 1$ in *LastColumn*)
bottom ← *LastToFirst*(position of *symbol* with rank $Count_{symbol}(bottom + 1, LastColumn)$ in *LastColumn*)

Note that these two lines of pseudocode merely compute the position of *symbol* with rank *i* in *FirstColumn* from its positions in *LastColumn*. This task can be compactly described without the First-to-Last mapping by the following two lines:

top ← position of *symbol* with rank $Count_{symbol}(top, LastColumn) + 1$ in *FirstColumn*
bottom ← position of *symbol* with rank $Count_{symbol}(bottom + 1, LastColumn)$ in *FirstColumn*

For *top* = 1, *bottom* = 6, and *symbol* = "n", the symbols "n" with ranks $Count_{\text{"n"}}(top, LastColumn) + 1 = 1$ and $Count_{\text{"a"}}(bottom + 1, LastColumn) = 3$ are located in positions 9 and 11 of *FirstColumn*.

STOP and Think: Do we need to store all of *FirstColumn* in memory in order to execute the preceding two lines of pseudocode?

Getting rid of the first column of the Burrows-Wheeler matrix

Define *FirstOccurrence*(*symbol*) as the first position of *symbol* in *FirstColumn*. If *Text* = `"panamabananas$"`, then *FirstColumn* is `"$aaaaaabmnnnps"`, and the array holding all values of *FirstOccurrence* is [0, 1, 7, 8, 9, 11, 12], as shown below. For DNA strings of any length, the array *FirstOccurrence* contains only five elements.

i	*FirstColumn*	FIRSTOCCURRENCE
0	$\$_1$	0
1	a_1	1
2	a_2	
3	a_3	
4	a_4	
5	a_5	
6	a_6	
7	b_1	7
8	m_1	8
9	n_1	9
10	n_2	
11	n_3	
12	p_1	12
13	s_1	13

The two lines of pseudocode from the previous step can now be rewritten as follows:

top ← *FirstOccurrence*(*symbol*) + $Count_{symbol}$(*top*, *LastColumn*)
bottom ← *FirstOccurrence*(*symbol*) + $Count_{symbol}$(*bottom* + 1, *LastColumn*) − 1

When *top* = 1, *bottom* = 6, and *symbol* = `"n"`, we have that *FirstOccurrence*(`"n"`) = 9, $Count_{\text{"n"}}$(*top*, *LastColumn*) = 0 and $Count_{\text{"n"}}$(*bottom* + 1, *LastColumn*) = 3, again implying that (*top* = 1, *bottom* = 6) will be updated as (*top* = 9 + 0 = 9, *bottom* = 9 + 3 − 1 = 11).

In the process of simplifying the green lines of pseudocode from **BWMATCHING**, we have also eliminated the need for both *FirstColumn* and *LastToFirst*, resulting in a more efficient algorithm called **BETTERBWMATCHING**.

```
BETTERBWMATCHING(FirstOccurrence, LastColumn, Pattern, Count)
  top ← 0
  bottom ← |LastColumn| − 1
  while top ≤ bottom
    if Pattern is nonempty
      symbol ← last letter in Pattern
      remove last letter from Pattern
      if positions from top to bottom in LastColumn contain an occurrence of symbol
        top ← FirstOccurrence(symbol) + Count_symbol(top, LastColumn)
        bottom ← FirstOccurrence(symbol) + Count_symbol(bottom + 1, LastColumn) − 1
      else
        return 0
    else
      return bottom − top + 1
```

You may be wondering why we called this algorithm "better" because if you have to compute the *Count* arrays as you go, then you will not obtain a runtime speedup. If you have to precompute these arrays, then you will need to store them in memory, which is space-intensive. Hold onto this thought.

CODE CHALLENGE: Implement **BETTERBWMATCHING**.

Input: A string *BWT*(*Text*) followed by a collection of strings *Patterns*.

Output: A list of integers, where the *i*-th integer corresponds to the number of substring matches of the *i*-th member of *Patterns* in *Text*.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/BetterBWMatching.txt)

Sample Input:

```
GGCGCCGC$TAGTCACACACGCCGTA
ACC CCG CAG
```

Sample Output:

```
1 2 1
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-301/step/7

We hope that you have noticed a serious limitation of **BETTERBWMATCHING** — even though this algorithm counts the number of occurrences of *Pattern* in *Text*, it fails to identify their starting positions! To locate pattern matches identified by **BETTERBWMATCHING**, we can once again use the suffix array, as shown in the figure below. For example, the suffix array immediately finds the three matches of `"ana"` in `"panamabananas$"`.

M(*Text*)	SUFFIXARRAY(*Text*)
$ p a n a m a b a n a n a s	13
a b a n a n a s $ p a n a m	5
a m a b a n a n a s $ p a n	3
a n a m a b a n a n a s $ p	**1**
a n a n a s $ p a n a m a b	**7**
a n a s $ p a n a m a b a n	**9**
a s $ p a n a m a b a n a n	11
b a n a n a s $ p a n a m a	6
m a b a n a n a s $ p a n a	4
n a m a b a n a n a s $ p a	2
n a n a s $ p a n a m a b a	8
n a s $ p a n a m a b a n a	10
p a n a m a b a n a n a s $	0
s $ p a n a m a b a n a n a	12

Figure: The suffixes of `"panamabananas$"` that begin the rows of *M*(`"panamabananas$"`) are highlighted, and the suffixes beginning with `"ana"` are shown in green. The suffix array records the starting position of each suffix in *Text*.

The suffix array makes our job easy, but recall that our original motivation for using the Burrows-Wheeler transform was to *reduce* the amount of memory used by the suffix array for pattern matching. If we add the suffix array to Burrows-Wheeler-based pattern matching, then we are right back where we started!

The memory-saving device that we will employ is inelegant but useful. We will build a **partial suffix array** of *Text*, denoted $SuffixArray_K(Text)$, which only contains values that are multiples of some positive integer K (see figure below). In real applications, partial suffix arrays are often constructed for $K = 100$, thus reducing memory usage by a factor of 100 compared to a full suffix array. See CHARGING STATION: Partial Suffix Array Construction (https://stepic.org/lesson/CS-Partial-Suffix-Array-Construction-9809) for more details.

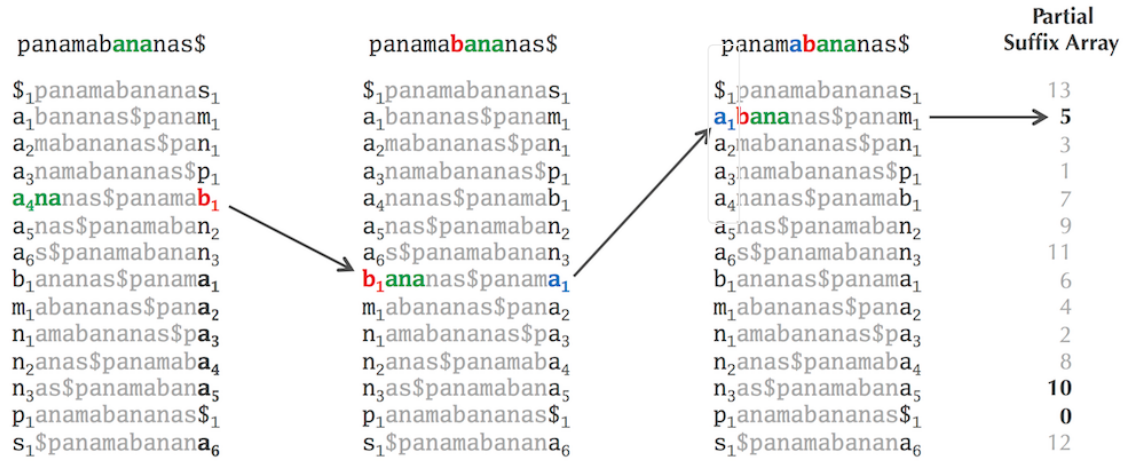

Figure: One of the matches of "ana" in "panamab**ana**nas$" is highlighted. This match occurs at starting position 7, but we don't immediately know this because we don't have the luxury of storing the entire suffix array. However, by walking backward, we find that "**ana**" is preceded by "$\mathbf{b_1}$", which in turn is preceded by "$\mathbf{a_1}$". The partial suffix array above, generated for $K = 5$, indicates that "$\mathbf{a_1}$" occurs at position 5 of "panama**bana**nas". Since it took us two steps to walk backward to "$\mathbf{a_1}$", we conclude that this occurrence of "ana" begins at position 5 + 2 = 7.

We will now discuss how to improve **BETTERBWMATCHING** (reproduced below) by resolving the trade-off between precomputing the values of $Count_{symbol}(i, LastColumn)$ (having to store them in memory) and computing these values as we go (requiring substantial runtime).

```
BETTERBWMATCHING(FirstOccurrence, LastColumn, Pattern, Count)
    top ← 0
    bottom ← |LastColumn| − 1
    while top ≤ bottom
        if Pattern is nonempty
            symbol ← last letter in Pattern
            remove last letter from Pattern
            if positions from top to bottom in LastColumn contain an occurrence of symbol
                top ← FirstOccurrence(symbol) + Count_symbol(top, LastColumn)
                bottom ← FirstOccurrence(symbol) + Count_symbol(bottom + 1, LastColumn) − 1
            else
                return 0
        else
            return bottom − top + 1
```

The balance that we strike is similar to the one used for the partial suffix array. Rather than storing $Count_{symbol}(i, LastColumn)$ for all positions i, we will only store the *Count* arrays when i is divisible by C, where C is a constant; these arrays are called **checkpoint arrays**. When C is large (C is typically equal to 100 in practice) and the alphabet is small (e.g., 4 nucleotides), checkpoint arrays require only a fraction of the memory used by *BWT*(*Text*).

i	*LastColumn*	COUNT						
		$	a	b	m	n	p	s
0	s_1	**0**	**0**	**0**	**0**	**0**	**0**	**0**
1	m_1	0	0	0	0	0	0	1
2	n_1	0	0	0	1	0	0	1
3	p_1	0	0	0	1	1	0	1
4	b_1	0	0	0	1	1	1	1
5	n_2	**0**	**0**	**1**	**1**	**1**	**1**	**1**
6	n_3	0	0	1	1	2	1	1
7	a_1	0	0	1	1	3	1	1
8	a_2	0	1	1	1	3	1	1
9	a_3	0	2	1	1	3	1	1
10	a_4	**0**	**3**	**1**	**1**	**3**	**1**	**1**
11	a_5	0	4	1	1	3	1	1
12	$\$_1$	0	5	1	1	3	1	1
13	a_6	1	5	1	1	3	1	1
		1	6	1	1	3	1	1

Figure: The *Count* checkpoint arrays for *Text* = `"panamabananas$"` and $C = 5$ are highlighted in bold. If we want to compute $Count_{\text{"a"}}$(13, `"smnpbnnaaaaa$a"`), then the checkpoint array at position 10 tells us that there are 3 occurrences of `"a"` before position 10 of `"smnpbnn`**`aaa`**`aa$a"`. We then check whether `"a"` is present position **10** (yes), **11** (yes), and **12** (no) of *LastColumn* to conclude that $Count_{\text{"a"}}$(13, `"smnpbnnaaaaa$a"`) = **3** + **2** = 5.

What about runtime? Using checkpoint arrays, we can compute the *top* and *bottom* pointers in a constant number of steps (i.e., fewer than *C*). Since each *Pattern* requires at most |*Pattern*| pointer updates, the modified **BETTERBWMATCHING** algorithm now requires O(|*Patterns*|) runtime, which is the same as using a trie or suffix array.

Furthermore, we now only have to store the following data in memory: *BWT*(*Text*), *FirstOccurrence*, the partial suffix array, and the checkpoint arrays. Storing this data requires memory approximately equal to 1.5 · |*Text*|. Thus, we have finally knocked down the memory required for solving the Multiple Pattern Matching Problem for millions of sequencing reads into a workable range.

CODE CHALLENGE: Solve the Multiple Pattern Matching Problem.

Input: A string *Text* followed by a collection of strings *Patterns*.

Output: All starting positions in *Text* where a string from *Patterns* appears as a substring.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/MultiplePatternMatching.txt)

Sample Input:

```
AATCGGGTTCAATCGGGGT
ATCG
GGGT
```

Sample Output:

```
1 4 11 15
```

Time Limit: `5 mins`

To solve this problem please visit stepic.org/lesson/-303/step/4

As the plummeting cost of reading short strings of DNA has made headlines, it is easy to fail to appreciate the progress that has been made in the computational side of read mapping. So, before continuing into approximate pattern matching, we would like to pause at the end of this chapter and reflect on just how far the algorithms for read mapping – and the hardware running them – have progressed. In 1975, the state-of-the-art Aho-Corasick algorithm was touted as requiring only 15 minutes to map just 24 English words to a dictionary. Just a generation later, when Burrows-Wheeler based approaches were introduced into read mapping, the same 15 minutes was used to map almost *ten million* reads to a reference genome. Having seen how far we have come in the last four decades, we can only imagine where it will take us in another four decades.

Reducing Approximate Pattern Matching to Exact Pattern Matching

In this section, we will return to the goal of identifying SNPs in an individual genome when compared against the reference genome. To do so, we need to generalize the Approximate Pattern Matching Problem from a previous chapter to the case of multiple patterns.

Multiple Approximate Pattern Matching Problem: *Find all approximate occurrences of a collection of patterns in a text.*

Input: A string *Text*, a collection *Patterns* of (shorter) strings, and an integer *d*.

Output: All starting positions in *Text* where a string from *Patterns* appears as a substring with at most *d* mismatches.

We begin with the simple observation that if *Pattern* occurs in *Text* with a single mismatch, then we can divide *Pattern* into two halves, one of which occurs exactly in *Text*, as illustrated below.

```
Pattern        acttggct
Text   ...ggcacactaggctcc...
```

Thus, we can find whether *Pattern* occurs with one mismatch in *Text* by dividing *Pattern* into two halves and then searching for exact matches of these smaller strings. If we find a match with one of these halves of *Pattern*, we then check if the entire string *Pattern* occurs with a single mutation.

This method can easily be generalized for approximate pattern matching with $d > 1$ mismatches; if *Pattern* approximately matches a substring of *Text* with at most d mismatches, then *Pattern* and *Text* must share at least one k-mer for a sufficiently large value of k. For example, if we are looking for a pattern of length 20 with at most $d = 3$ mismatches, then we can divide this pattern into four parts of length 20/(3+1)=5 and search for exact matches of these smaller substrings:

```
Pattern      acttaggctcgggataatcc
Text   ...actaagtctcgggataagcc...
```

This observation is helpful because it reduces *approximate* pattern matching to the *exact* matching of shorter patterns, which allows us to use fast algorithms that are designed for exact pattern matching but are not applicable to approximate pattern matching, such as approaches based on suffix trees and suffix arrays.

STOP and Think: If *Pattern* has length 23 and appears in *Text* with 3 mismatches, can we conclude that *Pattern* shares a 6-mer with *Text*? Can we conclude that it shares a 5-mer with *Text*?

The question remains how to find the maximum value of *k* that guarantees that any *Pattern* of length *n* with *d* mismatches in *Text* will have a *k*-mer substring exactly matching *Text*. We will employ the following theorem.

Theorem: If two strings of length *n* match with at most *d* mismatches, then they must share a *k*-mer of length $k = \lfloor n/(d+1) \rfloor$.

Proof: Divide the first string into *d* + 1 substrings, where the first *d* substrings have exactly *k* symbols and the final substring has at least *k* symbols. For the case *d* = 3, here is a subdivision of a 23-nucleotide string (*n* = 23) into *d* + 1 = 4 substrings, where the first 3 substrings have $k = \lfloor n/(d+1) \rfloor = 23/4 = 5$symbols and the last substring has 8 symbols.

acttaggctcgggataatccgga

If you distribute *d* mismatches among the string positions, then they may affect at most *d* substrings, which leaves at least one substring (of length at least *k*) unchanged. This substring must match the second string exactly, which establishes the theorem. □

We now have the outline of an algorithm for matching a string *Pattern* of length *n* to *Text* with at most *d* mismatches. We first divide *Pattern* into *d*+1 pieces of length $k = \lfloor n/(d+1) \rfloor$, called **seeds**. After finding which seeds match *Text* exactly (**seed detection**), we attempt to extend seeds in both directions in order to verify whether *Pattern* occurs in *Text* with at most *d* mismatches (**seed extension**).

Approximate Pattern Matching with the Burrows-Wheeler Transform

To extend the Burrows-Wheeler approach to approximate pattern matching, we will not stop when we encounter a mismatch. On the contrary, we will proceed onward until we either find an approximate match or exceed the limit of *d* mismatches.

The figure below illustrates the search for `"dna"` in `"panamabananas$"` with at most one mismatch. Let's first proceed as in the case of exact pattern matching, threading `"dna"` backwards using the Burrows-Wheeler transform. After finding six occurrences of `"a"`, we identify three exact occurrences of `"na"` and three inexact occurrences of `"na"`: `"pa"`, `"ma"`, and `"ba"`. We note that these three strings have accumulated a mismatch (keeping track of the number of mismatches in a separate column) and then continue with all six strings.

In the next step, none of the three exact matches of `"na"` can be extended into an exact match of `"dna"`, but all three can be extended into `"ana"`, which has one mismatch. We also fail to extend `"pa"`, `"ma"`, and `"ba"` into `"dpa"`, `"dma"`, and `"dba"`, respectively, and so we eliminate all three strings from consideration, resulting in only three approximate occurrences of `"dna"`.

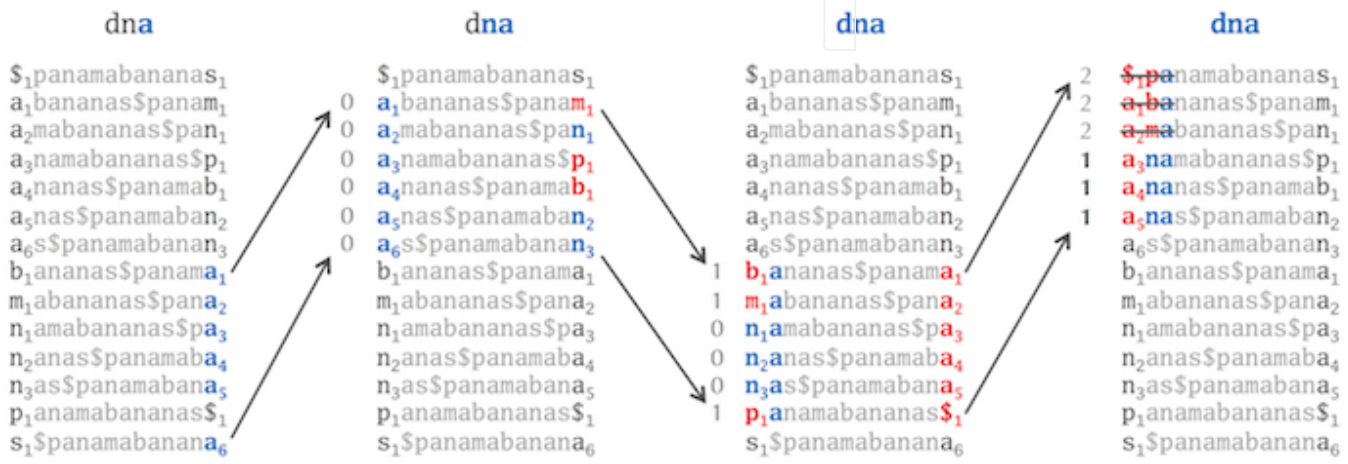

Figure: Using the Burrows-Wheeler transform to find approximate pattern matches of `"dna"` in `"panamabananas$"`. The number of mismatches encountered in a given row is shown in the column on the left.

In practice, this heuristic faces complications. We do not want to start allowing mismatched strings at the early stages of **BETTERBWMATCHING**, or else we will have to consider too many frivolous candidate strings. We may therefore require that a suffix of *Pattern* of some threshold length matches *Text* exactly. Moreover, the method becomes time-intensive when using large values of *d*, as we must explore many inexact matches. Practical applications often limit the value of *d* to at most 3.

You should now be ready to design your own approach to solve the Multiple Approximate Pattern Matching Problem and use this solution to map real sequencing reads.

CODE CHALLENGE: Solve the Multiple Approximate Pattern Matching Problem.

Input: A string *Text*, followed by a collection of strings *Patterns*, and an integer *d*.

Output: All positions where one of the strings in *Patterns* appears as a substring of *Text* with at most *d* mismatches.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/MultipleApproximatePatternMatching.txt)

Sample Input:

```
ACATGCTACTTT
ATT GCC GCTA TATT
1
```

Sample Output:

```
2 4 4 6 7 8 9
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-304/step/6

CHALLENGE PROBLEM: Given the Burrows-Wheeler transform and a partial suffix array of the bacterial genome *Mycoplasma pneumoniae* along with a collection of reads, find all reads occurring in the genome with at most 1 mismatch.

Download Dataset

To construct the partial suffix array $SuffixArray_k(Text)$, we first need to construct the full suffix array and then retain only the elements of this array that are divisible by K, along with their indices i. This is illustrated below for $Text$ = "panamabananas" and K = 5, where $SuffixArray_k(Text)$ corresponds to the bold elements.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
SUFFIXARRAY($Text$)	13	5	3	1	7	9	11	6	4	3	8	10	0	12

CODE CHALLENGE: Construct a partial suffix array.

Input: A string *Text* and a positive integer *K*.

Output: $SuffixArray_K(Text)$, in the form of a list of ordered pairs (*i*, *SuffixArray*(*i*)) for all nonempty entries in the partial suffix array.

Extra Dataset (/media/attachments/lessons/302/partialSuffixArray.txt)

Return to Main Text (https://stepic.org/lesson/Where-are-the-Matched-Patterns-302/step/2)

Sample Input:

```
PANAMABANANAS$
5
```

Sample Output:

```
1,5
11,10
12,0
```

Time Limit: `5 mins`

To solve this problem please visit stepic.org/lesson/-9809/step/2